PROJECT REPORT: SMART ROUTE OPTIMIZATION

Project Title - Smart Route Optimization

Course Code – CSE2001

Course Title – Fundamentals in AI & ML

Submitted By - DAKSH PATNI

Register Number - 25BAI10491

INTRODUCTION

Finding the shortest path between two locations is a fundamental problem in computer science and real-world applications such as navigation systems. Traditional algorithms like A* are widely used for this purpose.
This project focuses on implementing a shortest path finder using the A* algorithm. Additionally, a machine learning model is used to improve the heuristic function, making the algorithm more efficient.

PROBLEM STATEMENT

The objective of this project is to design and implement a system that finds the shortest path between two nodes in a graph without using Dijkstra's algorithm, while improving performance using machine learning.

FUNCTIONAL REQUIREMENTS

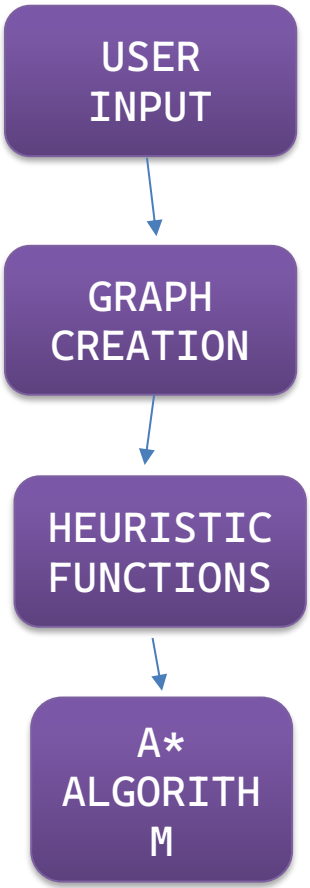
- Accept user input (start and destination nodes)
- Create a graph of nodes and edges
- Compute shortest path using A* algorithm

- Display path and total cost
- Visualize graph and shortest route

NON-FUNCTIONAL REQUIREMENTS

- System should be efficient and fast
- Code should be readable and modular
- Output should be accurate
- System should run on basic hardware
- User-friendly interface (CLI based)

SYSTEM ARCHITECHTURE

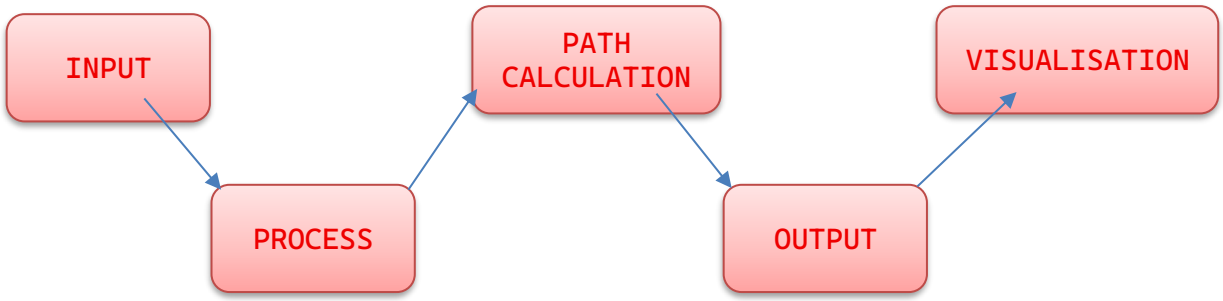


DESIGN DIAGRAMS

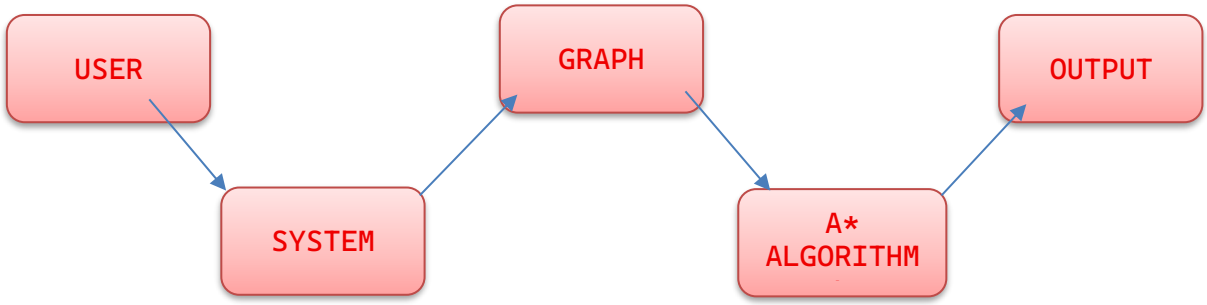
Use case Diagram:

- User interacts with system to input nodes and receive shortest paths.

Workflow Diagram:



Sequence Diagram:



Class Diagram:

- Graph
- Node
- Algorithm

- ML Model

DESIGN DECISIONS & RATIONALE

- A* algorithm chosen for efficiency over brute force methods.
- Euclidian distance used as heuristic.
- Machine learning added to improve heuristic prediction.
- NetworkX used for simplicity in graphic handling.

IMPLEMENTATION DETAILS

Language used: Python
Libraries used:

- NumPy
- NetworkX
- Matplotlib
- Scikit-learn

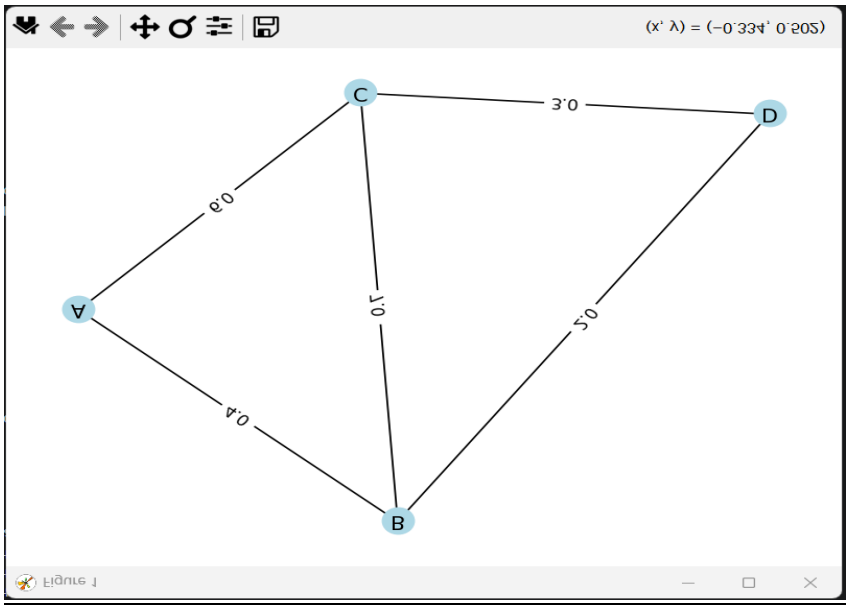
Modules used:

- Graph creation
- ML model training
- Pathfinding algorithm
- Visualisation

RESULTS

```
Enter cost from A to B:4
Enter cost from A to C:6
Enter cost from B to C:7
Enter cost from B to D:2
Enter cost from C to D:3
Choose start state:
1. A
2. B
3. C
4. D
A
Choose end state:
1. A
2. B
3. C
4. D
D
Path: ['A', 'B', 'C', 'D'], Cost: 14.0
Path: ['A', 'B', 'D'], Cost: 6.0
Path: ['A', 'C', 'B', 'D'], Cost: 15.0
Path: ['A', 'C', 'D'], Cost: 9.0

Shortest Path: ['A', 'B', 'D']
Minimum Cost: 6.0
```



TESTING APPROACH

- Tested with different node configurations.
- Checked invalid inputs.
- Compared results manually.
- Verified path correctness.

CHALLENGES FACED

- Understanding A* algorithm.
- Integrating ML with heuristic.
- Handling graph visualisation.
- Debugging incorrect paths.

KEY TAKEAWAYS

- Learned graph theory fundamentals.
- Understood shortest path algorithms.
- Gained experience in machine learning integration.
- Improved problem-solving skills.

FUTURE ENHACEMENTS

- Use real-world map data (GPS).
- Add traffic prediction.

- Build GUI application.
- Integrate with Google Maps API.

REFERECES

1. Introduction to Algorithm by Cormen.
2. Python Documentation.
3. NetworkX Documentation.
4. Scikit-learn Documentation.
5. Research papers on A* algorithm

CONCLUSION

This project successfully demonstrates the implementation of a shortest path finder using the A* algorithm. By incorporating machine learning into the heuristic function, the system becomes more efficient and adaptable. The project highlights the importance of combining classical algorithms with modern techniques for improved performance.